

# **Мастер-план тестирования (Master Test Plan)**

Проект: TIMP-project-client

Основан на стандарте ISO/IEC/IEEE 29119-3

Ответственный: Садыков Ростислав Андреевич

10 мая 2026 г.

## **Содержание**

|          |                                            |          |
|----------|--------------------------------------------|----------|
| <b>1</b> | <b>Введение</b>                            | <b>2</b> |
| <b>2</b> | <b>Объекты тестирования</b>                | <b>2</b> |
| <b>3</b> | <b>Функции, подлежащие тестированию</b>    | <b>2</b> |
| <b>4</b> | <b>Функции, НЕ подлежащие тестированию</b> | <b>2</b> |
| <b>5</b> | <b>Подход к тестированию</b>               | <b>2</b> |
| <b>6</b> | <b>Критерии прохождения</b>                | <b>2</b> |
| <b>7</b> | <b>Требования к тестовой среде</b>         | <b>3</b> |

# 1. Введение

Данный документ представляет собой формализованный план тестирования клиентской части (графического интерфейса и внутренней логики) проекта TIMP-project-client. Клиент реализован на C++ с использованием фреймворка Qt6 (приложение Qt Widgets). Цель тестирования – гарантировать корректную работу пользовательского интерфейса, навигации между окнами, надежность клиентского бэкенда и стабильность сетевых запросов.

## 2. Объекты тестирования

- **Модули графического интерфейса (UI):** MainWindow, AuthForm.
- **Клиентский бэкенд:** Класс FunctionsForClient (управление состояниями окон, обработка сигналов авторизации).

## 3. Функции, подлежащие тестированию

1. **Пользовательский интерфейс (UI):** Корректное переключение режимов видимости элементов, очистка памяти при смене окон.
2. **Бизнес-логика (FunctionsForClient):** Обработка логинов/паролей, генерация сигналов is\_auth, изоляция логики от отображения.
3. **Интеграция с сетью:** Отправка QString данных на сервер, обработка потери соединения.

## 4. Функции, НЕ подлежащие тестированию

- Стандартное поведение виджетов Qt (кнопки, поля ввода).
- Логика математических вычислений и работа БД (на стороне сервера).

## 5. Подход к тестированию

Общая стратегия строится на комбинированном подходе:

- **Unit-тестирование:** Использование фреймворка Qt Test для методов класса FunctionsForClient в изоляции от GUI.
- **Тестирование UI:** Проверка слотов и сигналов через имитацию событий Clicked.
- **Интеграционное тестирование:** Сквозной сценарий UI -> Signals -> TCP Socket -> Server.

## 6. Критерии прохождения

- **Pass:** 100% Unit-тестов успешны. Отсутствие утечек памяти (объекты UI корректно удаляются через delete). UI не блокирует Event Loop.
- **Fail:** Краш приложения, зависание интерфейса при недоступности сервера, утечки памяти при переключении форм.

## **7. Требования к тестовой среде**

ОС Windows/Linux, компиляторы MSVC/MinGW, Qt 6.x. Тестовый сервер-заглушка для обработки сетевых запросов.